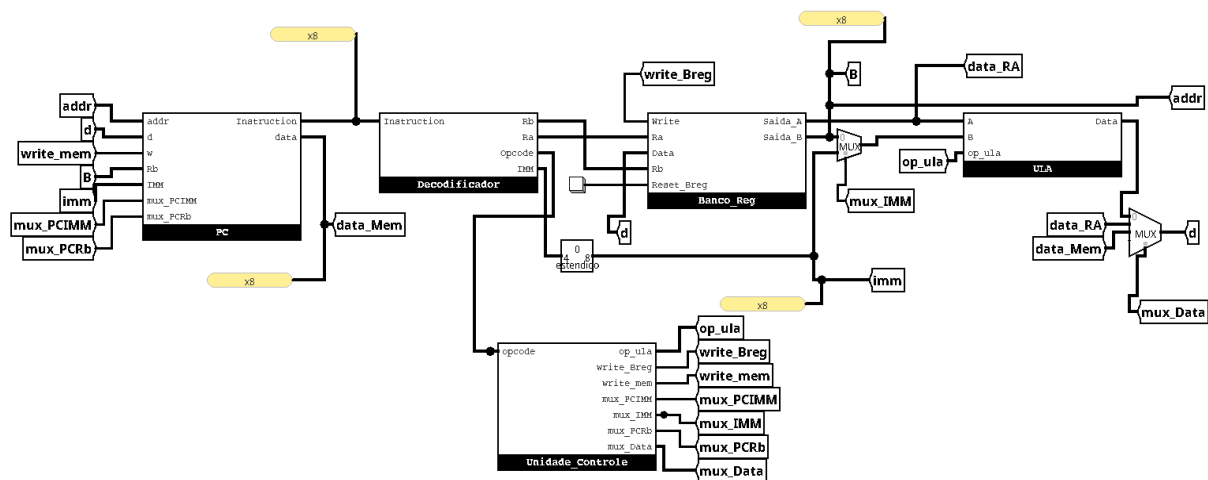


Relatório Trabalho 2 de Arquitetura de Computadores - Redux-V

Por Bruno Bertussi Camillo, GRR 20252947, 23/04/2026

1. Diagrama de Blocos do Datapath



A arquitetura implementada é do tipo monociclo, load-store e segue o modelo Von Neumann, partilhando a mesma memória para instruções e dados através do componente dual port ram. O caminho de dados foi projetado com os seguintes componentes principais:

- **Program Counter (PC):** Registo de 8 bits que armazena o endereço da próxima instrução.
- **Memória RAM (dual port ram):** Como é Von Neumann, o endereço de leitura da Porta 1 está ligado ao PC (para ir buscar a instrução) e o da Porta 2 está ligado à saída da ULA (para ler/escrever dados em operações Load/Store).
- **Banco de Registos:** Contém os 4 registos (r0 a r3). Possui duas portas de leitura e uma de escrita. Um multiplexador (MUX) na entrada de endereço de escrita garante que operações addi gravem sempre no r0.
- **Unidade Lógica e Aritmética (ULA):** Realiza as operações matemáticas e lógicas.
- **Unidade de Controle:** Decodifica os 4 bits do *Opcode* da instrução para gerar os sinais que controlam os multiplexadores, a ULA e as permissões de escrita.

2. Tabela de Códigos da ULA

A ULA foi desenhada para suportar todas as operações aritméticas e lógicas da especificação, bem como as novas instruções. Foi atribuído um código de controlo de 4 bits (*op_ula*) para cada operação interna:

OP_ULA	Operação
0000	A = 0
0001	not A
0010	A and B
0011	A or B
0100	A xor B
0101	A + B
0110	A - B
0111	A << 2^B
1000	A >> 2^B
1001	A != 0
1010	A++

3. Tabela de Controle

A Unidade de Controle principal traduz os 4 bits de Opcode nos seguintes sinais de controle para o Datapath:

opcode[3..0]	op_ula[3..0]	write_Breg	write_mem	mux_PCIMM	mux_IMM	mux_PCRb	mux_Data[1..0]
0000	0000	0	0	0	0	1	00
0001	0010	0	0	1	1	0	00
0010	0010	1	0	0	0	0	10
0011	0100	0	1	0	0	0	01
0100	0101	1	0	0	1	0	00
0101	1010	1	0	0	0	0	00
0110	1001	0	0	0	0	1	00
0111	1001	1	1	0	1	0	00
1000	0001	1	0	0	0	0	00
1001	0010	1	0	0	0	0	00
1010	0011	1	0	0	0	0	00
1011	0100	1	0	0	0	0	00
1100	0101	1	0	0	0	0	00
1101	0110	1	0	0	0	0	00
1110	0111	1	0	0	0	0	00
1111	1000	1	0	0	0	0	00

4. Justificativa para as Instruções Adicionadas

Para otimizar o programa desenvolvido no Trabalho 1 , foram utilizadas as 3 instruções cujos opcodes estavam reservados (0101, 0110, 0111):

1. **inc (Increment) - Opcode 0110:** No código original, o incremento dos iteradores (i++ ou k++) exigia a utilização do r0 com operações addi 1 seguidas de add r1, r0. Esta instrução permite incrementar diretamente qualquer registo em apenas 1 ciclo de clock, libertando o r0 e limpando a lógica do loop.
2. **brnz (Branch On Not Zero) - Opcode 0110:** O REDUX-V possui nativamente apenas o brzr, forçando a matemática a parar o loop apenas quando a diferença exata atinge zero. O brnz facilita o controle de loops regressivos ou comparações ativas, saltando para o endereço guardado em Rb sempre que Ra for diferente de zero.
3. **jr (Jump Register) - Opcode 0111:** A arquitetura sofria com uma forte limitação de distância: a instrução nativa ji (Jump Immediate) permitia saltar apenas 7 instruções de cada vez, devido ao tamanho do seu operando imediato. Isto tornava a navegação em programas maiores muito difíceis. Com o jr, basta guardar o endereço de destino num registo e instruir o Program Counter (PC) a assumir esse valor. Isto eliminou a restrição espacial, permitindo saltar para qualquer parte da memória num único ciclo.

5. Código Assembly Otimizado do Novo Programa

```
xor r1, r1    ; r1 (i) = 0

xor r2, r2    ; r2 (k) = 0

xor r0, r0

addi 4

xor r3, r3

add r3, r0    ; r3 = 4 (Início do Loop 1)

xor r0, r0

addi 5

add r0, r0

add r0, r0

add r0, r0

add r0, r0

addi 6

add r0, r1

st r2, r0    ; A[i] = k

inc r2       ; k++
```

```
xor r0, r0
addi 6
add r0, r0
add r0, r0
add r0, r0
add r0, r0
add r0, r1
st r2, r0 ; B[i] = k
inc r2 ; k++
inc r1 ; i++
xor r0, r0
addi 5
add r0, r0
sub r0, r1
brnz r0, r3 ; Volta para endereço 04 se r0 != 0
xor r1, r1 ; i = 0
xor r0, r0
addi 4
add r0, r0
add r0, r0
add r0, r0
addi 3
xor r3, r3
add r3, r0 ; r3 = 35 (Início do Loop 2)
xor r0, r0
addi 5
```

```
add r0, r0
add r0, r0
add r0, r0
add r0, r0
addi 6
add r0, r1
ld r2, r0    ; r2 = A[i]
xor r0, r0
addi 6
add r0, r0
add r0, r0
add r0, r0
add r0, r0
add r0, r1
ld r3, r0    ; r3 = B[i]
add r2, r3
xor r0, r0
addi 6
add r0, r0
add r0, r0
add r0, r0
add r0, r0
addi 5
addi 5
add r0, r1
st r2, r0    ; R[i] = soma
```

inc r1 ; i++

xor r0, r0

addi 5

add r0, r0

sub r0, r1

brnz r0, r3 ; Volta para endereço 35 se r0 != 0

xor r0, r0

addi 4

add r0, r0

add r0, r0

add r0, r0

add r0, r0

addi 5

addi 5

jr r0 ; Salto incondicional (Loop infinito)